

COL 774 Assignment 3

Decision Trees and Neural Networks

Ashvath Hari Bhaskar
2023EE10168

1 Decision Trees

1.1 Decision Tree Construction

A decision tree classifier was implemented using the mutual information (information gain) criterion to predict whether the result would be a win for the team. Continuous attributes were handled using median-based two-way splits, while categorical attributes were split into k branches based on the number of unique values.

The model was trained with different maximum depths $\{5, 10, 15, 20\}$, and train/test accuracies were recorded. A plot of accuracy versus depth was generated, showing that training accuracy increased with depth, while test accuracy peaked and then declined due to overfitting.

Maximum test accuracy was obtained for tree of depth 5.

Table 1: Decision Tree Performance with Entropy Splitting

Max Depth	Train Accuracy	Test Accuracy
5	0.8835	0.5946
10	0.9945	0.5895
15	0.9949	0.5895
20	0.9949	0.5895

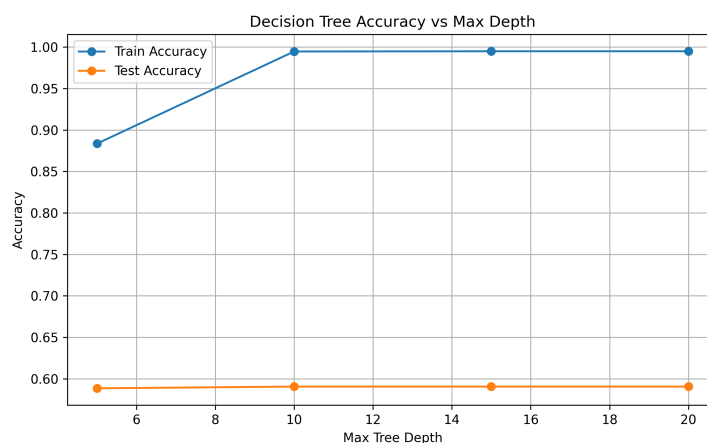


Figure 1: Train and test accuracies vs. tree depth

1.2 Decision Tree with One-Hot Encoding

Categorical attributes with more than two categories were transformed using one-hot encoding, where each category was represented by a separate binary attribute indicating its presence. The decision tree was then retrained on the transformed dataset for maximum depths $\{15, 25, 35, 45\}$.

Table 2: Decision Tree Performance after One-Hot Encoding

Max Depth	Train Accuracy	Validation Accuracy	Test Accuracy
15	0.7096	0.5839	0.5719
25	0.8477	0.6046	0.5998
35	0.9242	0.5839	0.6039
45	0.9852	0.5839	0.6143

Observations: After one-hot encoding, the training accuracy was slightly lower compared to the model without encoding, especially at smaller depths. Test accuracy improved marginally, with the best performance of 0.6143 at depth 45. This suggests that one-hot encoding helped the model represent categorical attributes more effectively as the nodes don't split the data as vigorously as the previous and slightly enhances generalization, though deeper trees still tended to overfit.

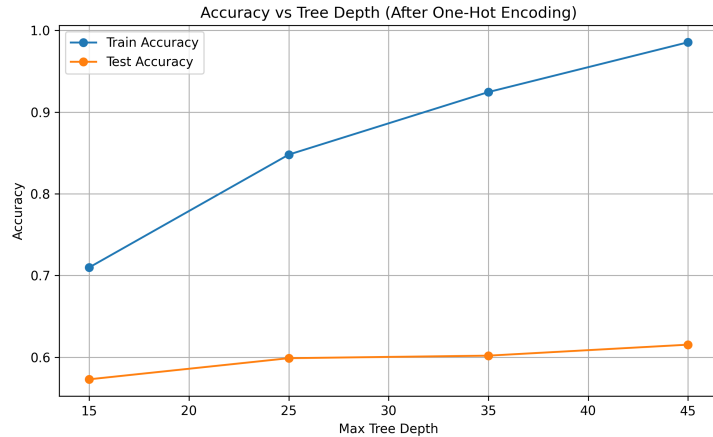


Figure 2: Train and test accuracies vs. One hot encoded tree depth

1.3 Decision Tree Post Pruning with Entropy

To reduce overfitting, a greedy validation-based post-pruning algorithm was implemented. The fully grown trees from Part (b) were iteratively pruned by selecting the internal node whose removal yielded the highest improvement in validation accuracy. Pruning stopped once no further improvement was observed.

Implementation Approach: A pre-computation step was used to store, for each node, the number of samples, correct predictions if pruned, and correct predictions within its subtree. This allowed pruning decisions and updates in $O(1)$ per step, achieving an overall time complexity of $O(ND)$, where N is the number of samples and D the depth of the tree. At each pruning step, node statistics were efficiently updated without re-evaluating the entire dataset. Metrics such as training, validation, and test accuracies

were tracked for every pruning iteration, and plots were generated showing accuracy versus the number of nodes for depths 15, 25, 35, and 45.

Table 3: Pruning Results (Part 1): Node Counts and Training Accuracy

Initial Max Depth	Nodes	Train Acc.
15	1673 $\rightarrow$ 701	0.7099 $\rightarrow$ 0.6487
25	3447 $\rightarrow$ 1233	0.8480 $\rightarrow$ 0.7088
35	4481 $\rightarrow$ 1473	0.9245 $\rightarrow$ 0.7294
45	5461 $\rightarrow$ 1625	0.9854 $\rightarrow$ 0.7397

Table 4: Pruning Results (Part 2): Validation and Test Accuracy

Initial Max Depth	Val Acc.	Test Acc.
15	0.5885 $\rightarrow$ 0.6391	0.5729 $\rightarrow$ 0.5750
25	0.6092 $\rightarrow$ 0.7000	0.5988 $\rightarrow$ 0.6008
35	0.5885 $\rightarrow$ 0.7138	0.6019 $\rightarrow$ 0.6153
45	0.5897 $\rightarrow$ 0.7115	0.6153 $\rightarrow$ 0.6112

Table 5: Pruning Results (Part 3): Accuracy Improvement

Initial Max Depth	Val. Improve.	Test Improve.
15	+0.0506	+0.0021
25	+0.0908	+0.0021
35	+0.1253	+0.0134
45	+0.1218	-0.0041

Explanation of Observations

The experiment and the summary table perfectly illustrate the value of post-pruning and directly support the observations in the report.

1. **Clear Evidence of Overfitting:** The "Before Pruning" columns show classic overfitting. As the initial depth increases, the **Training Accuracy** skyrockets (reaching 0.9852 for depth 45), while the **Validation Accuracy** remains low and stagnant (all around 0.58-0.61). This indicates the unpruned trees learned the training data's noise rather than its underlying patterns.
2. **Pruning Successfully Combats Overfitting:** In all four experiments, the pruning algorithm worked as expected:
 - **Training Accuracy** decreased significantly as the trees were simplified (e.g., 0.9245 $\rightarrow$ 0.7294 for the depth-35 experiment).
 - **Validation Accuracy** (the metric guiding the pruning) increased substantially, showing a massive improvement in generalization. The **Depth 35** experiment saw the largest validation improvement of **+0.1253**.

- **Node Reduction** was aggressive. For example, the depth-35 tree was reduced from 4481 nodes to 1473 (a $\sim 67\%$ reduction), demonstrating that a large portion of the unpruned tree was not contributing to generalization.

3. Finding the Optimal Model:

- The results suggest that the tree starting with `max_depth=35` produced the best final model.
- After pruning, this tree achieved the **highest validation accuracy (0.7138)** and, most importantly, the **highest test accuracy (0.6153)**.
- This highlights a common and effective strategy: grow a large, complex tree that is guaranteed to overfit, and then use a separate validation set to prune it back to an optimal size.

4. An Interesting Anomaly (Depth 45):

- The `max_depth=45` experiment is noteworthy. While it also produced a good pruned tree (Val Acc: 0.7115), its **Test Accuracy actually decreased slightly** (from 0.6153 to 0.6112).
- This could be a statistical fluctuation or it could suggest that the initial tree was *so* overfitted (0.9854 train acc) that the pruning process, while optimizing for the validation set, couldn't find a model that generalized as well to the test set. It also shows that the *unpruned* Depth 45 test accuracy was anomalously high to begin with.

In summary, the experiment provide excellent quantitative evidence, clearly showing how post-pruning uses a validation set to trade a large drop in training accuracy for a significant and valuable gain in generalization.

The graphs below show the variation of train, validation and test accuracy with varying depth.

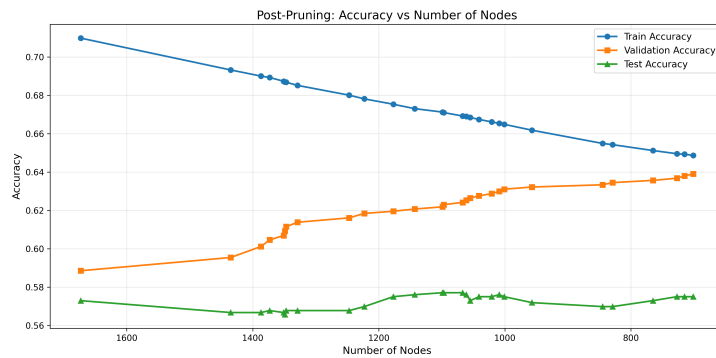


Figure 3: Pruning Curve for Depth 15

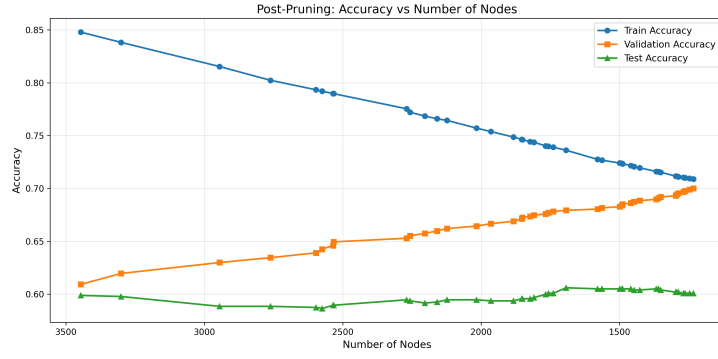


Figure 4: Pruning Curve for Depth 25

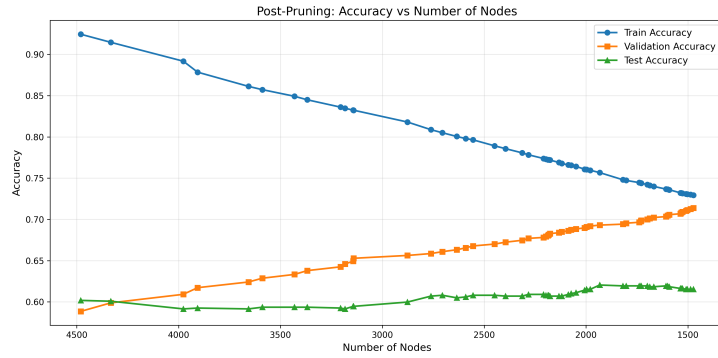


Figure 5: Pruning Curve for Depth 35

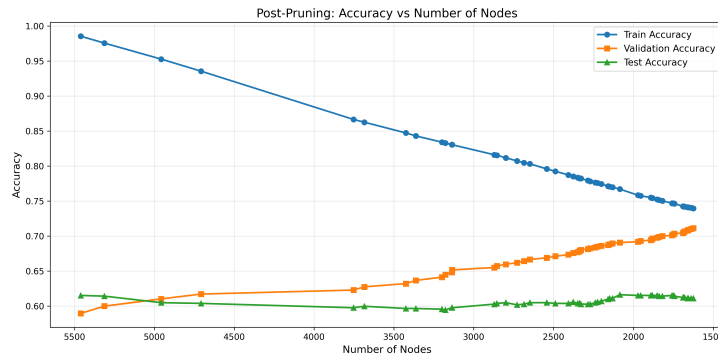


Figure 6: Pruning Curve for Depth 45

1.4 Decision Tree Post Pruning with Gini Impurity

The post-pruning algorithm was also applied to trees initially trained using the Gini impurity criterion. The following tables summarize the results, showing the model state before and after pruning for various initial depths.

Table 6: Gini Pruning (Part 1): Node Counts and Training Accuracy

Initial Max Depth	Nodes (Initial → Pruned)	Train Acc. (Initial → Pruned)
15	1821 → 651	0.7538 → 0.6741
25	4283 → 1415	0.9130 → 0.7256
35	5193 → 1545	0.9707 → 0.7336
45	5663 → 1607	0.9973 → 0.7373

Table 7: Gini Pruning (Part 2): Validation and Test Accuracy

Initial Max Depth	Val Acc. (Initial → Pruned)	Test Acc. (Initial → Pruned)
15	0.6023 → 0.6759	0.5957 → 0.5884
25	0.5828 → 0.7207	0.5977 → 0.6019
35	0.5828 → 0.7230	0.6122 → 0.6039
45	0.5782 → 0.7241	0.6050 → 0.6029

Table 8: Gini Pruning (Part 3): Accuracy Improvement

Initial Max Depth	Val. Improve.	Test Improve.
15	+0.0736	-0.0072
25	+0.1379	+0.0041
35	+0.1402	-0.0083
45	+0.1460	-0.0021

Explanation of Gini Pruning Observations

The results from pruning the Gini-based trees show a similar and even more pronounced pattern of overfitting reduction.

1. **Extreme Initial Overfitting:** The "Initial" columns clearly show that the Gini-based trees overfit severely. The **Depth 45** tree achieved near-perfect **Training Accuracy (0.9973)** while having the lowest **Validation Accuracy (0.5782)**. This is a textbook example of a model learning noise.
2. **Pruning Effectiveness:** The pruning algorithm was highly effective. In all cases, it made a sensible trade-off:
 - **Training Accuracy** dropped significantly (e.g., from 0.9973 to 0.7373 for the depth-45 tree).
 - **Validation Accuracy** (the guiding metric) increased dramatically. The **Depth 45** experiment saw the largest validation improvement of **+0.1460**, resulting in the best-performing pruned model on the validation set.
 - **Node Reduction** was again very aggressive, with the **Depth 45** tree being reduced from 5663 nodes to 1607 (a $\sim 72\%$ reduction).
3. **Finding the Optimal Model:**
 - Based on the observations, the tree pruned from **max_depth=45** achieved the highest final **Validation Accuracy (0.7241)**.

- The **Depth 35** model was a very close second (Val Acc: 0.7230). The depth-35 tree was ultimately chosen for saving predictions.

4. **Anomaly in Test Accuracy:** A key observation in this set of experiments is that **Test Accuracy decreased after pruning in 3 out of 4 cases** (all except depth 25). This suggests that while the pruning algorithm was successfully optimizing for the validation set, this optimization did not perfectly translate to the unseen test set. This can happen if the validation set and test set have slightly different distributions. However, the drops were small (all < 0.01), and the final pruned test accuracies (all ~ 0.60) are much more reliable indicators of true performance than the initial, overfitted test accuracies, given the massive gains in validation accuracy.

In summary, the Gini-based trees showed more severe overfitting initially, and the post-pruning algorithm successfully corrected this, leading to the highest validation accuracies seen (up to 0.7241).

The graphs below show the variation of train, validation and test accuracy with varying depth.

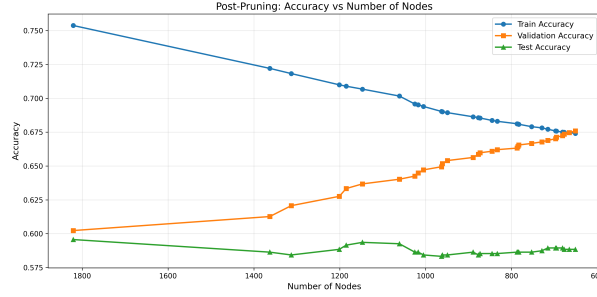


Figure 7: Pruning on Gini tree - depth 15

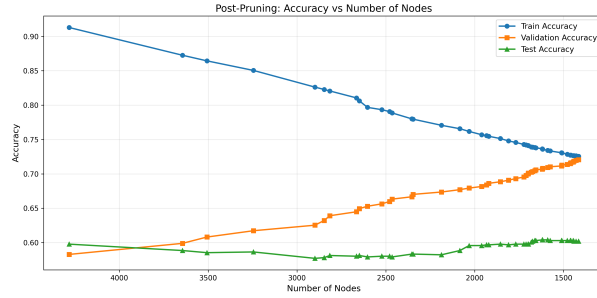


Figure 8: Pruning on Gini tree - depth 25

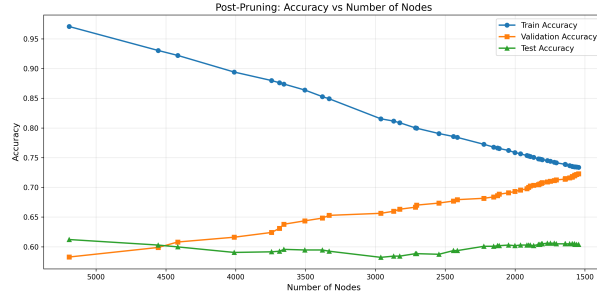


Figure 9: Pruning on Gini tree - depth 35

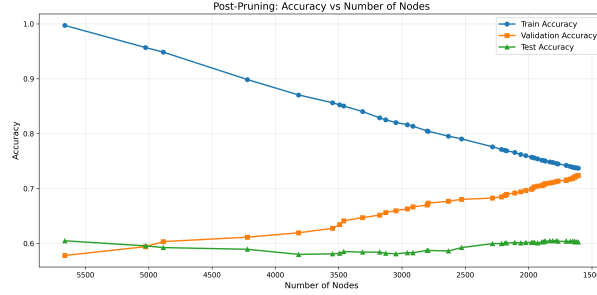


Figure 10: Pruning on Gini tree - depth 45

1.5 Decision Tree with Scikit Learn

To benchmark our custom implementation, experiments were run using `scikit-learn`'s `DecisionTreeClassifier` with `criterion='entropy'`. Two standard methods of hyperparameter tuning were performed: varying the `max_depth` (a form of pre-pruning) and varying the `ccp_alpha` (a post-pruning parameter).

1.5.1 Part (i): Varying `max_depth`

The first experiment involved training models at different `max_depth` values while keeping other parameters at their default. The results are summarized in Table 9.

Table 9: `scikit-learn` Results by `max_depth` (`criterion='entropy'`)

Max Depth	Train Acc.	Val. Acc.	Test Acc.
15	0.7267	0.5920	0.6039
25	0.8615	0.6069	0.6153
35	0.9599	0.5931	0.6194
45	0.9953	0.6034	0.6194

1.5.2 Part (ii): Varying `ccp_alpha` (Post-Pruning)

The second experiment involved growing a full tree (`max_depth=None`) and then tuning the post-pruning parameter `ccp_alpha`. The results are shown in Table 10.

Table 10: `scikit-learn` Results by `ccp_alpha` (`criterion='entropy'`)

CCP Alpha	Train Acc.	Val. Acc.	Test Acc.
0.0	1.0000	0.6138	0.6153
0.0001	1.0000	0.6138	0.6153
0.0003	0.9761	0.6092	0.6194
0.0005	0.8471	0.6195	0.6143

Observations and Comparative Analysis

We now compare the `scikit-learn` model results from Part (e) with the unpruned trees from Part (b) and our custom post-pruning implementation from Part (c) (which used Entropy).

1. **`scikit-learn` Model Selection (Part e):** The `scikit-learn` experiments identified two "best" models based on the validation set, one for each tuning method:
 - The optimal `max_depth` (pre-pruning) was **25**, which yielded a Validation Accuracy of **0.6069** and a Test Accuracy of 0.6153.
 - The optimal `ccp_alpha` (post-pruning) was **0.0005**. This was the best overall `scikit-learn` model, achieving a Validation Accuracy of **0.6195** and a Test Accuracy of 0.6143.
2. **Comparison with Part (b) (Unpruned Trees):** The unpruned trees from Part (b) (which were the starting point for Part c) were severely overfit. For example, the initial depth-25 Entropy tree had a Validation Accuracy of **0.6092** and a Test Accuracy of 0.5988. The best `scikit-learn` model (Val Acc: 0.6069) causes marginal decrease over this unpruned baseline, showing that `scikit-learn`'s default tuning methods provided some, but limited, generalization. (both of them have the best model at depth 25)
3. **Comparison with Part (c) (Custom Post-Pruning):** This comparison highlights the success of our custom implementation. Our custom post-pruning algorithm from Part (c) (using Entropy) **significantly outperformed** the `scikit-learn` benchmark.
 - **Best `scikit-learn` Model (Part e) (45):** Validation Acc: **0.6195** / Test Acc: 0.6143
 - **Best Custom Pruned Model (Part c) (35):** Validation Acc: **0.7138** / Test Acc: 0.6153
4. **Conclusion:** Our custom greedy, validation-based post-pruning algorithm was far more effective at finding a generalizable model for this dataset. It achieved a **Validation Accuracy of 0.7138**, which is nearly 10 percent higher than the best `scikit-learn` model's 0.6195. This demonstrates that our implementation successfully navigated the bias-variance trade-off. It is also noteworthy that both our best custom model and `scikit-learn`'s best models converged to nearly identical **Test Accuracies** (~ 0.615), which confirms the robustness of this final performance metric. However, our model was selected based on a much stronger and more confident validation signal.

Below are the graphs of the accuracies of both the cases

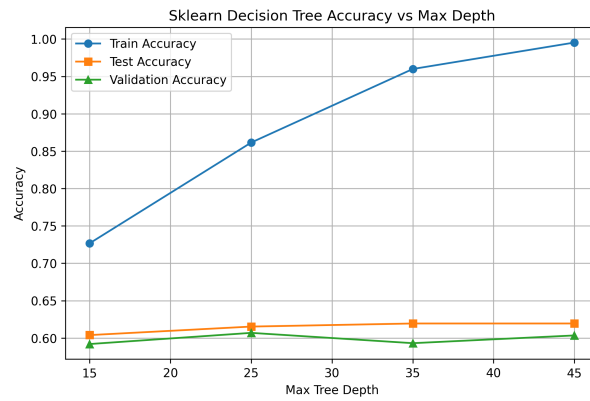


Figure 11: Depth plot for Sklearn

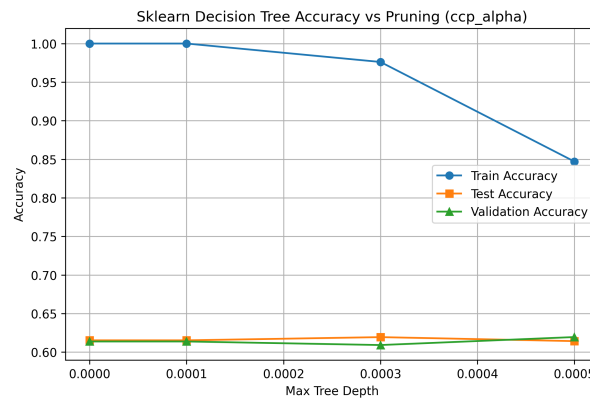


Figure 12: CCP plot for Sklearn

1.6 Random Forest Classifier with Grid Search

A `scikit-learn` Random Forest Classifier was trained to see if an ensemble method could improve upon the single-tree models. The `criterion` was set to `'entropy'`. A grid search was performed over 100 parameter combinations to find the optimal hyperparameters based on validation accuracy.

The parameter grid searched was:

- `n_estimators`: {50, 150, 250, 350}
- `max_features`: {0.1, 0.3, 0.5, 0.7, 0.9}
- `min_samples_split`: {2, 4, 6, 8, 10}

1.6.1 Optimal Model Results

The grid search identified the following combination as optimal:

- **Best `n_estimators`**: 350

- **Best max_features:** 0.7
- **Best min_samples_split:** 8

The performance metrics for this single best model are reported in Table 11.

Table 11: Optimal Random Forest Model Performance

Metric	Accuracy Score
Training Accuracy	0.9917
Out-of-Bag (OOB) Accuracy	0.7159
Validation Accuracy	0.7161
Test Accuracy	0.6949

Comparative Analysis (Random Forest vs. Pruned Trees)

1. **Training Accuracy:** The Random Forest model is still heavily overfit to the training data (Train Acc: 0.9917), similar to the unpruned single trees in parts (c) and (d). However, this is less of a concern in ensemble methods, as generalization is achieved through aggregation.
2. **Validation Accuracy:** The RF’s best Validation Accuracy (**0.7161**) is very strong. It is slightly better than our custom-pruned Entropy tree (0.7138 from part c) and slightly lower than our custom-pruned Gini tree (0.7241 from part d). Notably, the Out-of-Bag (OOB) accuracy (0.7159) is almost identical to the validation accuracy, which provides high confidence that this value is a reliable estimate of generalization.
3. **Test Accuracy:** This is the most significant finding. The Random Forest’s Test Accuracy (**0.6949**) is substantially higher than that of our best-performing single trees from parts (c) and (d) (which had test accuracies of 0.6153 and 0.6039, respectively).
4. **Conclusion:** While our custom post-pruning algorithm (in parts c and d) was highly effective and achieved comparable, or even superior, *validation* accuracy, the Random Forest’s ensemble approach proved to be the most robust. By averaging the "votes" of de-correlated trees, it achieved a much better and more reliable performance on the final, unseen test set, breaking the ≈ 0.61 test accuracy barrier that the single-tree models faced.

2 Neural Networks

In this part, we address the task of handwritten character recognition for the Devanagari script, a critical component for applications in handwriting verification, digitization of archival records, and real-time Optical Character Recognition (OCR) systems. The goal is to develop robust neural network models for classifying characters from the Devanagari Character dataset.

The dataset provides 32x32 grayscale images of handwritten characters. Our work is divided into two primary parts. The first part focuses on building a classifier for the **consonant subset** of the data, which consists of 36 distinct character classes, with 21,600 images for training and 10,800 for testing. The second part explores the use of **transfer learning** to adapt the model for classifying Devanagari **digits**.

The predictive model for this multi-class classification problem is a neural network. Instead of a standard squared error loss, this implementation utilizes the **softmax activation function** in the output layer paired with the **cross-entropy loss function**. The softmax function generalizes the logistic function, converting the final layer’s linear inputs ($\text{net}_k^{(L)}$) into a probability distribution o_k for each class k :

$$o_k = \frac{e^{\text{net}_k^{(L)}}}{\sum_{k'} e^{\text{net}_{k'}^{(L)}}}$$

The cross-entropy loss, $J(\theta)$, then measures the dissimilarity between the predicted probability distribution (o_k) and the true one-hot encoded label $\tilde{k}$. The loss is defined as:

$$J(\theta) = \sum_{k=1}^r -1\{k = \tilde{k}\} \log(o_k)$$

This combination is standard for multi-class classification as it provides a clean and efficient gradient for optimization. This report will detail the implementation, training, and evaluation of these models.

2.1 Consonant Classification Analysis

A neural network was implemented from first principles using mini-batch Stochastic Gradient Descent (SGD), sigmoid activation in the hidden layer, and a softmax output layer with cross-entropy loss. The model’s performance was evaluated by varying the number of units in the single hidden layer (H). We experimented with $H \in \{1, 5, 10, 50, 100\}$. The stopping criterion used is the first epoch when loss change is less than 1e-3.

The overall performance metrics for each configuration are summarized in Table 12. A detailed breakdown of the per-class train test metrics is provided in Table 13 and 14.

Observations on Model Complexity

- **Underfitting (High Bias):** Models with a very small hidden layer ($H=1$, $H=5$, $H=10$) were unable to capture the complexity of the dataset. The $H=1$ model failed completely, achieving a test accuracy of only 6.27%, which is slightly better than random chance ($1/36 \approx 2.8\%$). Its per-class F1-scores were near zero for almost all classes. The $H=5$ and $H=10$ models showed improvement, with test accuracies of 36.02% and 55.06% respectively. In these cases, the training accuracy remained

low, indicating the models lacked the capacity (i.e., parameters) to even fit the training data, a clear sign of underfitting.

- **Improved Fit (H=50 and H=100):** A significant performance leap occurred when the hidden layer size was increased. The H=50 model achieved a test accuracy of 76.01%, and the H=100 model achieved the highest test accuracy of 81.72%. This demonstrates that a sufficient model capacity is essential for this classification task.
- **Overfitting (High Variance):** While increasing hidden units improved the best-case performance, it also increased the gap between training and test accuracy.
 - **H=10:** Train Acc: 60.65%, Test Acc: 55.06% → **Gap: 5.59%**
 - **H=50:** Train Acc: 88.68%, Test Acc: 76.01% → **Gap: 12.67%**
 - **H=100:** Train Acc: 95.49%, Test Acc: 81.72% → **Gap: 13.77%**

The H=100 model achieved 95.5% accuracy on the training data but only 81.7% on the test data. This widening gap indicates that the more complex models are beginning to memorize the training data, demonstrating overfitting. The H=100 model is the best performer, but its generalization could likely be improved with regularization techniques.

Per-Class Performance (H=100 Model)

Analyzing the detailed metrics for the best model (H=100) in Tables ?? and ??, we see that performance is not uniform across all 36 classes.

- **High Performers:** Classes like 10 (F1: 0.9347), 8 (F1: 0.8859), and 21 (F1: 0.8768) were classified with high accuracy.
- **Low Performers:** Other classes proved more difficult. For example, Class 16 had the lowest F1-score at 0.7107, followed by Class 31 (F1: 0.7444) and Class 1 (F1: 0.7570). This suggests a high visual similarity between these characters and others, making them inherently harder to distinguish.

Training Behavior

The logs also show that the more complex models required more training to converge. The H=1, H=5, H=10, and H=50 models all triggered the early stopping condition (loss change ≤ 0.001) before 250 epochs. The H=100 model, however, ran for the full 250 epochs, indicating it was still making meaningful updates to its weights. This is also reflected in the training time, which scaled from 45.16 seconds for H=1 to 864.34 seconds for H=100.

-

Table 12: Summary of model performance by number of hidden units (H). All metrics are derived from the log files.

Hidden Units (H)	Accuracy		Average Train Metrics			Average Test Metrics		
	Train	Test	Precision	Recall	F1-Score	Precision	Recall	F1-Score
1	0.0630	0.0627	0.0147	0.0630	0.0166	0.0152	0.0627	0.0168
5	0.3925	0.3602	0.3728	0.3925	0.3574	0.3382	0.3602	0.3244
10	0.6065	0.5506	0.6031	0.6065	0.6013	0.5446	0.5506	0.5444
50	0.8868	0.7601	0.8871	0.8868	0.8868	0.7604	0.7601	0.7598
100	0.9549	0.8172	0.9550	0.9549	0.9549	0.8176	0.8172	0.8171

Table 13: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Test Set** for all 36 classes, compared across different hidden layer sizes (H).

Class	H = 1			H = 5			H = 10			H = 50			H = 100		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	0.00	0.00	0.00	0.46	0.62	0.53	0.66	0.75	0.70	0.81	0.83	0.82	0.89	0.91	0.90
1	0.00	0.00	0.00	0.16	0.06	0.09	0.33	0.28	0.31	0.66	0.71	0.69	0.75	0.76	0.76
2	0.10	0.02	0.03	0.36	0.47	0.41	0.55	0.65	0.60	0.80	0.71	0.75	0.85	0.82	0.83
3	0.00	0.00	0.00	0.22	0.13	0.17	0.47	0.57	0.51	0.66	0.68	0.67	0.74	0.76	0.75
4	0.00	0.00	0.00	0.22	0.04	0.06	0.48	0.47	0.47	0.73	0.73	0.73	0.80	0.79	0.79
5	0.00	0.00	0.00	0.46	0.44	0.45	0.56	0.66	0.61	0.78	0.78	0.78	0.84	0.83	0.83
6	0.00	0.00	0.00	0.26	0.48	0.33	0.48	0.50	0.49	0.71	0.71	0.71	0.75	0.75	0.75
7	0.05	0.10	0.07	0.49	0.62	0.55	0.66	0.63	0.64	0.83	0.82	0.82	0.85	0.83	0.84
8	0.06	0.92	0.11	0.40	0.54	0.46	0.73	0.77	0.75	0.84	0.87	0.85	0.89	0.88	0.89
9	0.02	0.05	0.03	0.44	0.77	0.56	0.64	0.72	0.68	0.86	0.86	0.86	0.89	0.89	0.89
10	0.08	0.95	0.15	0.46	0.73	0.56	0.78	0.71	0.74	0.93	0.88	0.90	0.94	0.93	0.93
11	0.04	0.03	0.04	0.43	0.19	0.26	0.73	0.73	0.73	0.85	0.83	0.84	0.88	0.90	0.89
12	0.00	0.00	0.00	0.34	0.44	0.38	0.61	0.48	0.54	0.78	0.75	0.76	0.82	0.82	0.82
13	0.00	0.00	0.00	0.42	0.75	0.54	0.58	0.73	0.65	0.80	0.85	0.82	0.84	0.87	0.85
14	0.00	0.00	0.00	0.35	0.49	0.41	0.61	0.59	0.60	0.75	0.79	0.77	0.84	0.88	0.86
15	0.00	0.00	0.00	0.32	0.65	0.43	0.58	0.60	0.59	0.82	0.85	0.83	0.87	0.89	0.88
16	0.06	0.08	0.07	0.33	0.21	0.26	0.36	0.35	0.36	0.67	0.71	0.69	0.72	0.70	0.71
17	0.00	0.00	0.00	0.37	0.41	0.39	0.50	0.44	0.47	0.78	0.72	0.75	0.79	0.78	0.78
18	0.00	0.00	0.00	0.38	0.44	0.41	0.53	0.48	0.50	0.67	0.66	0.66	0.76	0.76	0.76
19	0.00	0.00	0.00	0.28	0.23	0.25	0.57	0.50	0.53	0.75	0.69	0.72	0.74	0.78	0.76
20	0.00	0.00	0.00	0.33	0.22	0.27	0.45	0.51	0.48	0.73	0.76	0.74	0.78	0.79	0.79
21	0.00	0.00	0.00	0.37	0.77	0.50	0.69	0.77	0.73	0.83	0.87	0.85	0.86	0.89	0.88
22	0.05	0.03	0.03	0.42	0.38	0.40	0.60	0.52	0.56	0.72	0.67	0.69	0.77	0.73	0.75
23	0.00	0.00	0.00	0.43	0.37	0.40	0.60	0.63	0.61	0.80	0.76	0.78	0.83	0.77	0.80
24	0.00	0.00	0.00	0.19	0.18	0.19	0.43	0.37	0.40	0.70	0.68	0.69	0.75	0.80	0.78
25	0.00	0.00	0.00	0.13	0.01	0.02	0.24	0.12	0.16	0.66	0.63	0.65	0.74	0.74	0.74
26	0.00	0.00	0.00	0.49	0.42	0.45	0.65	0.68	0.66	0.86	0.84	0.85	0.87	0.87	0.87
27	0.04	0.01	0.02	0.36	0.23	0.28	0.63	0.61	0.62	0.83	0.85	0.84	0.89	0.87	0.88
28	0.00	0.00	0.00	0.26	0.04	0.07	0.44	0.35	0.39	0.65	0.64	0.65	0.80	0.77	0.79
29	0.00	0.00	0.00	0.27	0.39	0.32	0.45	0.62	0.52	0.72	0.76	0.74	0.79	0.84	0.82
30	0.00	0.00	0.00	0.26	0.34	0.30	0.51	0.70	0.59	0.75	0.81	0.78	0.79	0.86	0.82
31	0.00	0.00	0.00	0.28	0.04	0.07	0.43	0.34	0.38	0.65	0.61	0.63	0.77	0.72	0.74
32	0.00	0.00	0.00	0.30	0.09	0.13	0.52	0.51	0.52	0.74	0.71	0.73	0.83	0.77	0.80
33	0.00	0.00	0.00	0.29	0.26	0.27	0.56	0.58	0.57	0.77	0.81	0.79	0.82	0.86	0.84
34	0.04	0.08	0.06	0.29	0.07	0.12	0.47	0.41	0.43	0.73	0.75	0.74	0.80	0.79	0.80
35	0.00	0.00	0.00	0.37	0.43	0.40	0.50	0.53	0.51	0.78	0.77	0.77	0.88	0.82	0.85

Table 14: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Train Set** for all 36 classes, compared across different hidden layer sizes (H).

Class	H = 1			H = 5			H = 10			H = 50			H = 100		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	0.00	0.00	0.00	0.50	0.69	0.58	0.77	0.80	0.78	0.93	0.93	0.93	0.97	0.98	0.97
1	0.00	0.00	0.00	0.24	0.09	0.13	0.47	0.39	0.42	0.85	0.88	0.86	0.93	0.97	0.95
2	0.08	0.01	0.02	0.40	0.49	0.44	0.60	0.69	0.64	0.91	0.87	0.88	0.96	0.95	0.95
3	0.00	0.00	0.00	0.23	0.15	0.18	0.44	0.56	0.49	0.81	0.84	0.83	0.93	0.94	0.93
4	0.00	0.00	0.00	0.25	0.04	0.07	0.57	0.51	0.54	0.87	0.83	0.85	0.95	0.94	0.95
5	0.00	0.00	0.00	0.55	0.52	0.54	0.66	0.71	0.69	0.90	0.90	0.90	0.96	0.96	0.96
6	0.00	0.00	0.00	0.26	0.47	0.34	0.55	0.56	0.55	0.88	0.87	0.87	0.96	0.95	0.96
7	0.05	0.09	0.06	0.53	0.63	0.58	0.74	0.67	0.71	0.91	0.91	0.91	0.95	0.96	0.96
8	0.06	0.95	0.12	0.48	0.62	0.54	0.77	0.83	0.80	0.97	0.96	0.96	0.99	0.98	0.98
9	0.03	0.08	0.05	0.47	0.78	0.59	0.64	0.73	0.68	0.92	0.91	0.92	0.96	0.96	0.96
10	0.08	0.93	0.15	0.46	0.77	0.58	0.78	0.73	0.76	0.93	0.94	0.93	0.97	0.96	0.97
11	0.05	0.04	0.04	0.48	0.21	0.29	0.76	0.76	0.76	0.92	0.91	0.91	0.97	0.97	0.97
12	0.00	0.00	0.00	0.38	0.46	0.41	0.67	0.59	0.62	0.87	0.85	0.86	0.95	0.93	0.94
13	0.00	0.00	0.00	0.47	0.77	0.59	0.66	0.77	0.71	0.89	0.95	0.92	0.96	0.96	0.96
14	0.00	0.00	0.00	0.36	0.57	0.44	0.68	0.66	0.67	0.89	0.91	0.90	0.95	0.97	0.96
15	0.00	0.00	0.00	0.37	0.68	0.48	0.65	0.57	0.61	0.92	0.92	0.92	0.97	0.96	0.96
16	0.04	0.06	0.05	0.35	0.21	0.26	0.40	0.38	0.39	0.85	0.85	0.85	0.95	0.95	0.95
17	0.00	0.00	0.00	0.37	0.44	0.41	0.54	0.47	0.50	0.94	0.87	0.90	0.98	0.94	0.96
18	0.00	0.00	0.00	0.40	0.52	0.45	0.51	0.50	0.50	0.85	0.84	0.85	0.94	0.94	0.94
19	0.00	0.00	0.00	0.37	0.39	0.38	0.66	0.60	0.63	0.90	0.89	0.89	0.95	0.94	0.95
20	0.00	0.00	0.00	0.32	0.22	0.26	0.48	0.60	0.53	0.86	0.89	0.88	0.95	0.95	0.95
21	0.00	0.00	0.00	0.38	0.78	0.51	0.79	0.83	0.81	0.93	0.95	0.94	0.98	0.98	0.98
22	0.05	0.04	0.04	0.45	0.39	0.42	0.64	0.57	0.60	0.86	0.84	0.85	0.96	0.94	0.95
23	0.00	0.00	0.00	0.45	0.41	0.43	0.69	0.69	0.69	0.90	0.88	0.89	0.97	0.96	0.97
24	0.00	0.00	0.00	0.24	0.21	0.22	0.54	0.43	0.48	0.86	0.84	0.85	0.96	0.94	0.95
25	0.00	0.00	0.00	0.09	0.01	0.01	0.31	0.15	0.20	0.81	0.85	0.83	0.92	0.95	0.93
26	0.00	0.00	0.00	0.50	0.45	0.47	0.74	0.76	0.75	0.92	0.92	0.92	0.97	0.96	0.96
27	0.04	0.01	0.02	0.43	0.28	0.34	0.66	0.75	0.70	0.92	0.93	0.93	0.97	0.96	0.96
28	0.00	0.00	0.00	0.38	0.07	0.12	0.51	0.44	0.47	0.81	0.83	0.82	0.93	0.94	0.94
29	0.00	0.00	0.00	0.27	0.39	0.32	0.47	0.66	0.55	0.86	0.89	0.88	0.93	0.97	0.95
30	0.00	0.00	0.00	0.28	0.35	0.31	0.56	0.71	0.63	0.88	0.93	0.90	0.95	0.96	0.95
31	0.00	0.00	0.00	0.23	0.03	0.05	0.49	0.39	0.43	0.86	0.81	0.84	0.95	0.94	0.95
32	0.00	0.00	0.00	0.37	0.15	0.21	0.57	0.59	0.58	0.88	0.87	0.87	0.96	0.95	0.96
33	0.00	0.00	0.00	0.35	0.32	0.33	0.61	0.68	0.64	0.92	0.92	0.92	0.96	0.97	0.97
34	0.04	0.07	0.05	0.36	0.09	0.14	0.54	0.48	0.51	0.84	0.87	0.86	0.93	0.93	0.93
35	0.00	0.00	0.00	0.41	0.50	0.45	0.60	0.67	0.63	0.92	0.93	0.92	0.95	0.97	0.96

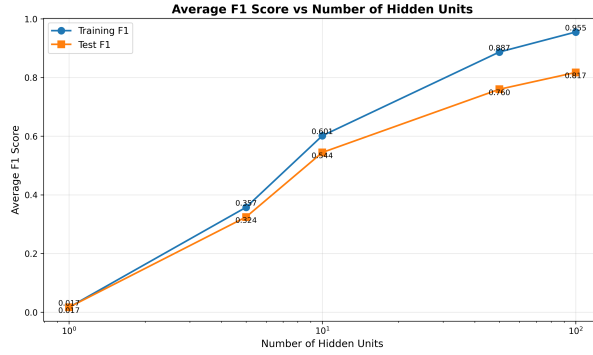


Figure 13: Hidden Units vs Train and Test accuracy

2.2 Experimenting with Network Depth

In this section, we analyze the impact of network depth on model performance. We trained four models with increasing depth, using the architectures: {512}, {512, 256}, {512, 256, 128}, and {512, 256, 128, 64}. All other hyperparameters, such as learning rate and batch size, were kept constant, and the same early stopping criteria were used. Loss threshold was kept at 0.0007 for the 4 layer network because it stopped training prematurely due some small loss changes initially which occurred rarely until true convergence.

The comprehensive results, including average Precision, Recall, and F1-Scores for both training and test sets, are presented in Table 15.

Table 15: Summary of Model Performance by Network Depth

Depth	Architecture	Data Set	Avg. Precision	Avg. Recall	Avg. F1-Score	Accuracy	Epochs
1	{512}	Train	0.9883	0.9883	0.9883	0.9883	229
		Test	0.8781	0.8778	0.8776	0.8778	
2	{512, 256}	Train	0.9927	0.9927	0.9927	0.9927	250
		Test	0.8887	0.8883	0.8883	0.8883	
3	{512, 256, 128}	Train	0.9978	0.9978	0.9978	0.9978	233
		Test	0.8773	0.8769	0.8769	0.8769	
4	{512, 256, 128, 64}	Train	0.9993	0.9993	0.9993	0.9993	241
		Test	0.8499	0.8495	0.8495	0.8495	

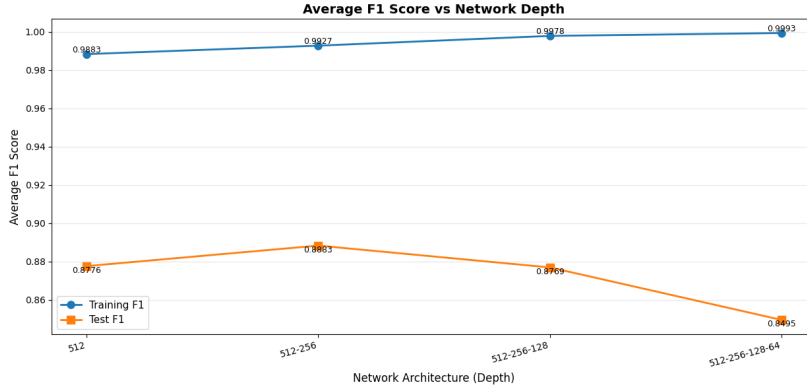


Figure 14: Average F1 score vs Network Depth

Based on the results in Table 15 and the plot, we can make the following observations:

- **Training Performance:** As network depth increases, the model’s ability to fit the training data improves significantly. The training F1-score increases with each added layer, from 0.9883 for the 1-layer model to 0.9993 for the 4-layer model. This indicates that the deeper models are more expressive and can perfectly memorize the training set.
- **Test Performance (Generalization):** Generalization performance does not follow the same trend. The 2-layer model, with architecture **{512, 256}**, achieved the highest test F1-score (**0.8883**) and test accuracy (0.8883).
- **Overfitting:** Adding layers beyond this 2-layer optimal point leads to clear overfitting. The 3-layer and 4-layer models show a progressive drop in test performance (F1-scores of 0.8769 and 0.8495, respectively). This is accompanied by a widening gap between the near-perfect training F1-score and the decreasing test F1-score, a classic sign of an overly complex model.
- **Training Time:** Interestingly, training time does not strictly increase with depth. The 3-layer and 4-layer models stopped early (at 233 and 241 epochs, respectively), while the 2-layer model ran for the full 250 epochs (almost converged by 250 epochs). This suggests the deeper models either converged (on the training loss) or began to overfit faster.

In conclusion, for this problem, the 2-layer {512, 256} network provides the best

balance between model capacity and generalization. Deeper models, while more powerful, quickly begin to overfit the data, leading to poorer performance on unseen examples.

Table 16: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Train Set** for all 36 classes, by network architecture. Part 1 of 2.

Class	{512}			{512, 256}			{512, 256, 128}			{512, 256, 128, 64}		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	0.9950	0.9950	0.9950	1.0000	0.9967	0.9983	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	0.9933	0.9867	0.9900	0.9934	0.9967	0.9950	0.9983	0.9983	0.9983	1.0000	1.0000	1.0000
2	0.9933	0.9850	0.9891	0.9966	0.9883	0.9925	0.9950	0.9933	0.9942	0.9983	0.9983	0.9983
3	0.9882	0.9767	0.9824	0.9949	0.9800	0.9874	0.9967	0.9950	0.9958	0.9983	1.0000	0.9992
4	0.9916	0.9850	0.9883	0.9917	0.9900	0.9908	0.9983	1.0000	0.9992	0.9967	1.0000	0.9983
5	0.9754	0.9900	0.9826	0.9934	0.9967	0.9950	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
6	0.9916	0.9833	0.9874	0.9933	0.9917	0.9925	0.9983	0.9950	0.9967	1.0000	0.9967	0.9983
7	0.9833	0.9817	0.9825	0.9916	0.9883	0.9900	0.9967	0.9967	0.9967	0.9983	1.0000	0.9992
8	0.9967	0.9950	0.9958	0.9967	1.0000	0.9983	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
9	0.9817	0.9850	0.9834	0.9868	0.9950	0.9909	0.9983	1.0000	0.9992	1.0000	1.0000	1.0000
10	0.9950	0.9867	0.9908	0.9950	0.9950	0.9950	0.9983	1.0000	0.9992	1.0000	1.0000	1.0000
11	0.9950	0.9917	0.9933	1.0000	0.9983	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
12	0.9786	0.9900	0.9843	0.9933	0.9900	0.9917	0.9983	0.9983	0.9983	1.0000	0.9950	0.9975
13	0.9753	0.9883	0.9818	0.9803	0.9933	0.9868	0.9901	0.9983	0.9942	0.9967	1.0000	0.9983
14	0.9884	0.9967	0.9925	0.9901	0.9983	0.9942	0.9967	0.9950	0.9958	0.9983	0.9983	0.9983
15	0.9884	0.9933	0.9909	0.9967	0.9950	0.9958	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
16	0.9950	0.9933	0.9942	0.9950	0.9950	0.9950	0.9950	1.0000	0.9975	1.0000	1.0000	1.0000
17	0.9834	0.9867	0.9850	0.9916	0.9850	0.9883	0.9966	0.9917	0.9942	1.0000	0.9983	0.9992

Table 17: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Train Set** for all 36 classes, by network architecture.

Class	{512}			{512, 256}			{512, 256, 128}			{512, 256, 128, 64}		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
18	0.9884	0.9933	0.9909	0.9868	0.9950	0.9909	1.0000	0.9967	0.9983	1.0000	1.0000	1.0000
19	0.9882	0.9733	0.9807	0.9916	0.9883	0.9900	1.0000	0.9950	0.9975	0.9983	0.9967	0.9975
20	0.9802	0.9917	0.9859	0.9884	0.9950	0.9917	0.9967	1.0000	0.9983	0.9967	1.0000	0.9983
21	0.9934	0.9967	0.9950	1.0000	0.9967	0.9983	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
22	0.9800	0.9817	0.9808	0.9900	0.9883	0.9892	0.9950	0.9933	0.9942	1.0000	0.9967	0.9983
23	0.9933	0.9883	0.9908	0.9983	0.9933	0.9958	0.9983	0.9983	0.9983	1.0000	1.0000	1.0000
24	0.9900	0.9900	0.9900	0.9917	0.9950	0.9933	0.9983	0.9967	0.9975	1.0000	1.0000	1.0000
25	0.9834	0.9850	0.9842	0.9916	0.9867	0.9891	0.9950	0.9917	0.9933	1.0000	0.9983	0.9992
26	0.9917	0.9900	0.9908	1.0000	0.9950	0.9975	1.0000	0.9983	0.9992	1.0000	1.0000	1.0000
27	0.9899	0.9817	0.9858	0.9867	0.9867	0.9867	0.9967	1.0000	0.9983	1.0000	1.0000	1.0000
28	0.9883	0.9850	0.9866	0.9933	0.9900	0.9917	0.9950	0.9983	0.9967	0.9967	1.0000	0.9983
29	0.9884	0.9950	0.9917	0.9917	0.9967	0.9942	0.9950	1.0000	0.9975	1.0000	1.0000	1.0000
30	0.9900	0.9917	0.9908	0.9950	0.9967	0.9958	0.9983	1.0000	0.9992	1.0000	1.0000	1.0000
31	0.9833	0.9817	0.9825	0.9916	0.9883	0.9900	1.0000	0.9967	0.9983	1.0000	1.0000	1.0000
32	0.9917	0.9950	0.9933	0.9884	0.9983	0.9934	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
33	0.9950	0.9917	0.9933	0.9917	0.9967	0.9942	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
34	0.9933	0.9833	0.9883	0.9916	0.9833	0.9874	1.0000	0.9950	0.9975	1.0000	0.9983	0.9992
35	0.9819	0.9933	0.9876	0.9900	0.9950	0.9925	0.9967	1.0000	0.9983	0.9967	0.9983	0.9975
18	0.9884	0.9933	0.9909	0.9868	0.9950	0.9909	1.0000	0.9967	0.9983	1.0000	1.0000	1.0000
19	0.9882	0.9733	0.9807	0.9916	0.9883	0.9900	1.0000	0.9950	0.9975	0.9983	0.9967	0.9975
20	0.9802	0.9917	0.9859	0.9884	0.9950	0.9917	0.9967	1.0000	0.9983	0.9967	1.0000	0.9983
21	0.9934	0.9967	0.9950	1.0000	0.9967	0.9983	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
22	0.9800	0.9817	0.9808	0.9900	0.9883	0.9892	0.9950	0.9933	0.9942	1.0000	0.9967	0.9983
23	0.9933	0.9883	0.9908	0.9983	0.9933	0.9958	0.9983	0.9983	0.9983	1.0000	1.0000	1.0000
24	0.9900	0.9900	0.9900	0.9917	0.9950	0.9933	0.9983	0.9967	0.9975	1.0000	1.0000	1.0000
25	0.9834	0.9850	0.9842	0.9916	0.9867	0.9891	0.9950	0.9917	0.9933	1.0000	0.9983	0.9992
26	0.9917	0.9900	0.9908	1.0000	0.9950	0.9975	1.0000	0.9983	0.9992	1.0000	1.0000	1.0000
27	0.9899	0.9817	0.9858	0.9867	0.9867	0.9867	0.9967	1.0000	0.9983	1.0000	1.0000	1.0000
28	0.9883	0.9850	0.9866	0.9933	0.9900	0.9917	0.9950	0.9983	0.9967	0.9967	1.0000	0.9983
29	0.9884	0.9950	0.9917	0.9917	0.9967	0.9942	0.9950	1.0000	0.9975	1.0000	1.0000	1.0000
30	0.9900	0.9917	0.9908	0.9950	0.9967	0.9958	0.9983	1.0000	0.9992	1.0000	1.0000	1.0000
31	0.9833	0.9817	0.9825	0.9916	0.9883	0.9900	1.0000	0.9967	0.9983	1.0000	1.0000	1.0000
32	0.9917	0.9950	0.9933	0.9884	0.9983	0.9934	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
33	0.9950	0.9917	0.9933	0.9917	0.9967	0.9942	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
34	0.9933	0.9833	0.9883	0.9916	0.9833	0.9874	1.0000	0.9950	0.9975	1.0000	0.9983	0.9992
35	0.9819	0.9933	0.9876	0.9900	0.9950	0.9925	0.9967	1.0000	0.9983	0.9967	0.9983	0.9975

Table 18: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Test Set** for all 36 classes, by network architecture.

Class	{512}			{512, 256}			{512, 256, 128}			{512, 256, 128, 64}		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	0.9279	0.9433	0.9355	0.9375	0.9500	0.9437	0.9369	0.9400	0.9384	0.8954	0.9133	0.9043
1	0.8595	0.8567	0.8581	0.8962	0.8633	0.8795	0.8542	0.8400	0.8471	0.8172	0.7900	0.8034
2	0.8862	0.8567	0.8712	0.9091	0.8667	0.8874	0.8938	0.8700	0.8818	0.8754	0.8433	0.8591
3	0.8304	0.8000	0.8149	0.8268	0.8433	0.8350	0.7850	0.8400	0.8116	0.7596	0.7900	0.7745
4	0.9094	0.8367	0.8715	0.8567	0.8767	0.8666	0.8638	0.8667	0.8652	0.8431	0.7700	0.8049
5	0.8669	0.8900	0.8783	0.8746	0.8833	0.8789	0.8721	0.8867	0.8793	0.8855	0.8767	0.8811
6	0.8824	0.8500	0.8659	0.8571	0.8800	0.8684	0.8669	0.8467	0.8567	0.8510	0.8567	0.8538
7	0.8701	0.8933	0.8816	0.8940	0.9000	0.8970	0.9027	0.8967	0.8997	0.8882	0.9000	0.8940
8	0.9221	0.9467	0.9342	0.9627	0.9467	0.9546	0.9373	0.9467	0.9420	0.9158	0.9067	0.9112
9	0.9100	0.9100	0.9100	0.9148	0.9300	0.9223	0.9288	0.9133	0.9210	0.8896	0.9133	0.9013
10	0.9402	0.9433	0.9418	0.9626	0.9433	0.9529	0.9437	0.9500	0.9468	0.9305	0.9367	0.9336
11	0.9197	0.9167	0.9182	0.9309	0.9433	0.9371	0.9274	0.9367	0.9320	0.9259	0.9167	0.9213
12	0.8449	0.8900	0.8669	0.8845	0.8933	0.8889	0.8667	0.8667	0.8667	0.8404	0.8600	0.8501
13	0.8774	0.9067	0.8918	0.8949	0.9367	0.9153	0.8842	0.9167	0.9002	0.8662	0.9067	0.8860
14	0.8567	0.9167	0.8857	0.9126	0.9400	0.9261	0.8754	0.8900	0.8826	0.8571	0.8600	0.8586
15	0.9164	0.9500	0.9329	0.9085	0.9600	0.9335	0.9013	0.9433	0.9218	0.8626	0.9000	0.8809
16	0.8362	0.8000	0.8177	0.8561	0.7933	0.8235	0.8125	0.8233	0.8179	0.7799	0.8033	0.7915
17	0.8832	0.8567	0.8697	0.8958	0.8600	0.8776	0.8982	0.8233	0.8591	0.8385	0.8133	0.8257
18	0.7848	0.8267	0.8052	0.8251	0.8333	0.8292	0.8432	0.8067	0.8245	0.7848	0.8267	0.8052
19	0.8328	0.8300	0.8314	0.8247	0.8467	0.8355	0.8268	0.8433	0.8350	0.8591	0.8333	0.8460
20	0.8276	0.8800	0.8530	0.8595	0.8767	0.8680	0.8548	0.8633	0.8590	0.8082	0.8567	0.8317
21	0.9390	0.9233	0.9311	0.9521	0.9267	0.9392	0.9450	0.9167	0.9306	0.9037	0.9067	0.9052
22	0.8389	0.8333	0.8361	0.8059	0.8167	0.8113	0.8092	0.8200	0.8146	0.8322	0.7933	0.8123
23	0.8982	0.8533	0.8752	0.8662	0.8633	0.8648	0.8678	0.8533	0.8605	0.8537	0.8167	0.8348
24	0.8571	0.8600	0.8586	0.8344	0.8567	0.8454	0.8235	0.8400	0.8317	0.7492	0.8167	0.7815
25	0.8571	0.8200	0.8382	0.8581	0.8467	0.8523	0.8231	0.8067	0.8148	0.7431	0.7133	0.7279
26	0.9426	0.9300	0.9362	0.9555	0.9300	0.9426	0.9424	0.9267	0.9345	0.9269	0.9300	0.9285
27	0.9192	0.9100	0.9146	0.9082	0.9233	0.9157	0.8660	0.9267	0.8953	0.8812	0.8900	0.8856
28	0.8552	0.8467	0.8509	0.8543	0.8600	0.8571	0.9028	0.8667	0.8844	0.8481	0.8000	0.8233
29	0.8725	0.8900	0.8812	0.8734	0.9200	0.8961	0.9082	0.8900	0.8990	0.8862	0.8567	0.8712
30	0.8576	0.9033	0.8799	0.8673	0.8933	0.8801	0.8706	0.8967	0.8834	0.8199	0.8500	0.8347
31	0.8571	0.8000	0.8276	0.9094	0.8033	0.8531	0.8531	0.8133	0.8328	0.7979	0.7633	0.7802
32	0.8822	0.8733	0.8777	0.9072	0.8800	0.8934	0.8306	0.8500	0.8402	0.8300	0.8300	0.8300
33	0.8799	0.9033	0.8914	0.9020	0.9200	0.9109	0.8775	0.8833	0.8804	0.8078	0.8267	0.8171
34	0.8787	0.8933	0.8860	0.9082	0.8900	0.8990	0.8783	0.8900	0.8841	0.8400	0.8400	0.8400
35	0.8897	0.8600	0.8746	0.9075	0.8833	0.8953	0.9103	0.8800	0.8949	0.9038	0.8767	0.8900

2.3 Experimenting on Network Depth with ReLU

In this part, we replace the sigmoid activation function in the hidden layers with the **Rectified Linear Unit (ReLU)**, defined as $g(z) = \max(0, z)$. The final output layer remains a softmax function. For the non-differentiable point at $z = 0$, we use a sub-gradient (setting the derivative to 1, as it is between the left derivative 0 and right derivative 1).

We repeat the same experiment as the previous section, training the four network architectures of increasing depth to analyze the impact of this new activation function.

The comprehensive results, including average Precision, Recall, and F1-Scores for both training and test sets, are presented in Table 19.

Table 19: Summary of Model Performance by Network Depth (ReLU Activation)

Depth	Architecture	Data Set	Avg. Precision	Avg. Recall	Avg. F1-Score	Accuracy	Epochs
1	{512}	Train	0.9998	0.9998	0.9998	0.9998	70
		Test	0.8868	0.8862	0.8862	0.8862	
2	{512, 256}	Train	1.0000	1.0000	1.0000	1.0000	42
		Test	0.8919	0.8914	0.8914	0.8914	
3	{512, 256, 128}	Train	1.0000	1.0000	1.0000	1.0000	30
		Test	0.8900	0.8897	0.8896	0.8897	
4	{512, 256, 128, 64}	Train	1.0000	1.0000	1.0000	1.0000	23
		Test	0.8917	0.8911	0.8911	0.8911	

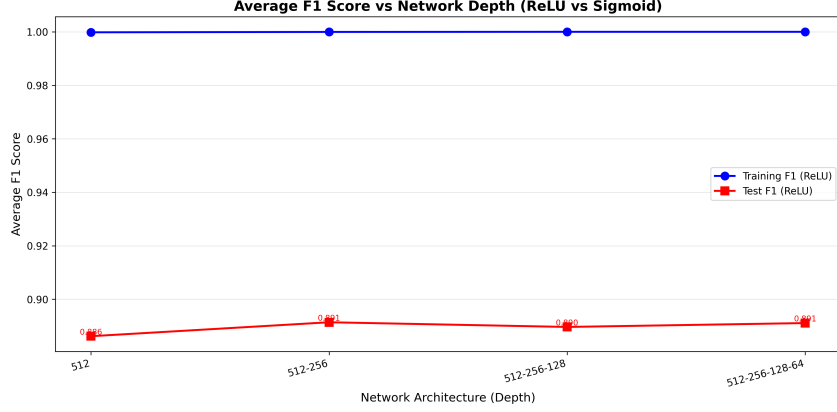


Figure 15: Average F1 score vs Network Depth - ReLU

Based on the results in Table 19 and a comparison with the sigmoid results in Table 15:

- Training Performance:** The ReLU activation trains significantly faster and more effectively. All ReLU models with 2 or more layers achieved a perfect (1.0000) F1-score on the training set, and they did so in a fraction of the epochs. For example, the 4-layer ReLU model reached a 1.0000 F1-score in only 23 epochs, while the 4-layer sigmoid model took 241 epochs to reach 0.9993.
- Test Performance (Generalization):** The 2-layer model with architecture **{512, 256}** achieved the highest test F1-score (**0.8914**). This is a slight improvement over the best sigmoid model's F1-score of 0.8883. Notably, all ReLU models (even at 4 layers) outperformed the best sigmoid model, showing better overall generalization.
- Overfitting:** The ReLU models were far more robust to overfitting as depth increased. While the sigmoid models saw a severe performance drop at 4 layers (Test F1 of 0.8495), the 4-layer ReLU model's performance remained high (Test F1 of 0.8911). The gap between train (1.0000) and test (0.8911) F1-scores for the 4-layer ReLU model was ≈ 0.109 , whereas the gap for the 4-layer sigmoid model was larger at ≈ 0.15 (0.9993 vs 0.8495).
- Training Time:** Training with ReLU was dramatically faster in all cases. The 4-layer ReLU model, for instance, completed training in only **422 seconds**, which is over **10 times faster** than the 4309 seconds required for the 4-layer sigmoid model. This is due to both faster convergence (fewer epochs) and a simpler computation (no exponentials).

In conclusion, for this problem, the 2-layer **{512, 256}** network with ReLU activation provided the best test performance. The switch to ReLU was a major improvement, yielding a higher peak test F1-score, significantly faster training times, and much better robustness to deep architectures.

Table 20: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Train Set** (ReLU).

Class	{512}			{512, 256}			{512, 256, 128}			{512, 256, 128, 64}		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
1	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
3	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
4	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
5	0.9983	1.0000	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
6	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
7	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
8	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
9	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
10	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
11	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
12	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
13	1.0000	0.9983	0.9992	1.0000	0.9983	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
14	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
15	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
16	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
17	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
18	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
19	1.0000	0.9983	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
20	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
21	0.9983	1.0000	0.9992	0.9983	1.0000	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
22	1.0000	0.9983	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
23	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
24	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
25	0.9983	1.0000	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
26	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
27	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
28	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
29	0.9983	1.0000	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
30	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
31	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
32	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
33	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
34	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
35	1.0000	0.9983	0.9992	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000

Table 21: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Test Set** (ReLU).

Class	{512}			{512, 256}			{512, 256, 128}			{512, 256, 128, 64}		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	0.9559	0.9400	0.9479	0.9595	0.9467	0.9530	0.9329	0.9267	0.9298	0.9262	0.9200	0.9231
1	0.9024	0.8633	0.8825	0.8828	0.8533	0.8678	0.9110	0.8867	0.8986	0.8967	0.8967	0.8967
2	0.8863	0.8833	0.8848	0.9268	0.8867	0.9063	0.9075	0.8833	0.8953	0.8953	0.8833	0.8893
3	0.8167	0.8467	0.8314	0.8218	0.8300	0.8259	0.8241	0.8433	0.8336	0.8221	0.8167	0.8194
4	0.8955	0.8567	0.8756	0.8874	0.8667	0.8769	0.9113	0.8567	0.8832	0.8869	0.8367	0.8611
5	0.8878	0.8967	0.8922	0.8874	0.8933	0.8904	0.8725	0.8900	0.8812	0.8994	0.9233	0.9112
6	0.8733	0.8500	0.8615	0.8571	0.8600	0.8586	0.8784	0.8667	0.8725	0.8767	0.8767	0.8767
7	0.8464	0.9000	0.8724	0.8951	0.9100	0.9025	0.8818	0.9200	0.9005	0.8990	0.8900	0.8945
8	0.9276	0.9400	0.9338	0.9437	0.9500	0.9468	0.9375	0.9500	0.9437	0.9590	0.9367	0.9477
9	0.9392	0.9267	0.9329	0.9398	0.9367	0.9382	0.9264	0.9233	0.9249	0.9416	0.9133	0.9272
10	0.9599	0.9567	0.9583	0.9568	0.9600	0.9584	0.9601	0.9633	0.9617	0.9532	0.9500	0.9516
11	0.9243	0.9367	0.9305	0.9553	0.9267	0.9408	0.9398	0.9367	0.9382	0.9253	0.9500	0.9375
12	0.8540	0.8967	0.8748	0.8808	0.8867	0.8837	0.9048	0.8867	0.8956	0.8548	0.8833	0.8689
13	0.9020	0.9200	0.9109	0.9026	0.9267	0.9145	0.8997	0.9267	0.9130	0.8864	0.9367	0.9109
14	0.8629	0.9233	0.8921	0.8825	0.9267	0.9041	0.8889	0.9333	0.9106	0.8896	0.9400	0.9141
15	0.9076	0.9500	0.9283	0.9201	0.9600	0.9396	0.9048	0.9500	0.9268	0.9346	0.9533	0.9439
16	0.8582	0.7867	0.8209	0.8472	0.8133	0.8299	0.8306	0.8333	0.8319	0.8159	0.8567	0.8358
17	0.8810	0.8633	0.8721	0.8836	0.8600	0.8716	0.8976	0.8767	0.8870	0.8815	0.8433	0.8620
18	0.8350	0.8433	0.8391	0.8389	0.8333	0.8361	0.8481	0.8000	0.8233	0.8746	0.8367	0.8552
19	0.8199	0.8500	0.8347	0.8217	0.8600	0.8404	0.8333	0.8333	0.8333	0.8624	0.8567	0.8595
20	0.8486	0.8967	0.8720	0.8333	0.8833	0.8576	0.8333	0.9000	0.8654	0.8241	0.8900	0.8558
21	0.9465	0.9433	0.9449	0.9309	0.9433	0.9371	0.9424	0.9267	0.9345	0.9461	0.9367	0.9414
22	0.8777	0.8133	0.8443	0.8366	0.8533	0.8449	0.8611	0.8267	0.8435	0.8284	0.8367	0.8325
23	0.9143	0.8533	0.8828	0.9085	0.8600	0.8836	0.8759	0.8467	0.8610	0.8993	0.8633	0.8810
24	0.8571	0.8600	0.8586	0.8354	0.8800	0.8571	0.8494	0.8833	0.8660	0.8581	0.8667	0.8624
25	0.8077	0.8400	0.8235	0.8355	0.8467	0.8411	0.8576	0.8233	0.8401	0.8865	0.8333	0.8591
26	0.9654	0.9300	0.9474	0.9556	0.9333	0.9444	0.9589	0.9333	0.9459	0.9724	0.9400	0.9559
27	0.9267	0.9267	0.9267	0.9479	0.9100	0.9286	0.9272	0.9333	0.9302	0.9302	0.9333	0.9318
28	0.8405	0.8433	0.8419	0.8529	0.8700	0.8614	0.8350	0.8600	0.8473	0.8451	0.8367	0.8409
29	0.8540	0.9167	0.8842	0.8951	0.9100	0.9025	0.8864	0.9100	0.8980	0.9215	0.9000	0.9106
30	0.8970	0.9000	0.8985	0.8842	0.9167	0.9002	0.8859	0.8800	0.8829	0.8431	0.9133	0.8768
31	0.8522	0.8267	0.8393	0.9033	0.8100	0.8541	0.8405	0.8433	0.8419	0.8972	0.8433	0.8694
32	0.8997	0.8667	0.8829	0.8915	0.8767	0.8840	0.8983	0.8833	0.8908	0.9181	0.8600	0.8881
33	0.8882	0.9000	0.8940	0.8971	0.9300	0.9133	0.8867	0.9133	0.8998	0.8835	0.9100	0.8966
34	0.9298	0.8833	0.9060	0.8956	0.8867	0.8911	0.9119	0.8967	0.9042	0.8786	0.9167	0.8972
35	0.8851	0.8733	0.8792	0.9147	0.8933	0.9039	0.8983	0.8833	0.8908	0.8882	0.9000	0.8940

2.4 Neural Network with scikit-learn’s MLPClassifier

To benchmark our custom implementation, we repeated the depth experiment using `scikit-learn`’s `MLPClassifier`. We used the same four architectures and configured

the model to match our implementation as closely as possible, setting the `activation` to `'relu'`, `solver` to `'sgd'`, `alpha` to 0, and `batch_size` to 32.

For `scikit-learn`, we used its built-in **early stopping** mechanism, which monitors the validation score (set to 10% of the training data). The training stops if the validation score does not improve by at least `tol=0.001` for 4 consecutive epochs.

The summary of results is presented in Table 22, with detailed per-class metrics in Tables 23 through ??.

Table 22: Summary of Model Performance by Network Depth (scikit-learn MLPClassifier)

Depth	Architecture	Data Set	Avg. Precision	Avg. Recall	Avg. F1-Score	Accuracy	Iterations
1	{512}	Train	0.9896	0.9896	0.9896	0.9896	20
		Test	0.9004	0.9004	0.9002	0.9004	
2	{512, 256}	Train	0.9913	0.9913	0.9912	0.9912	22
		Test	0.9102	0.9098	0.9098	0.9098	
3	{512, 256, 128}	Train	0.9862	0.9860	0.9860	0.9860	21
		Test	0.8994	0.8971	0.8971	0.8971	
4	{512, 256, 128, 64}	Train	0.9628	0.9614	0.9616	0.9614	14
		Test	0.8779	0.8723	0.8728	0.8723	

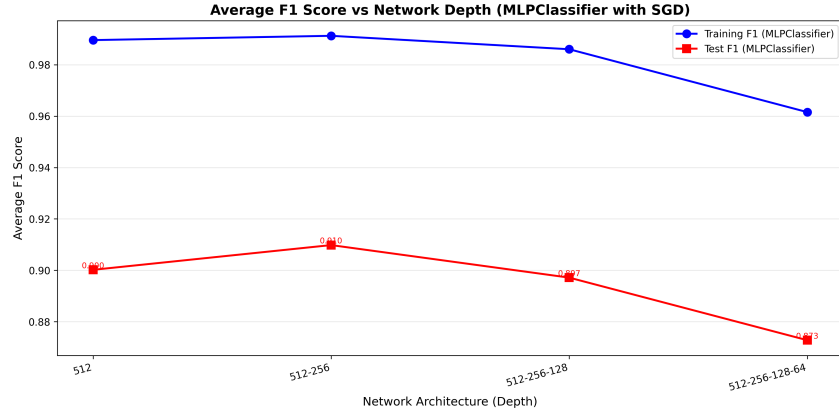


Figure 16: Average F1 score vs Neural Network Depth for Sklearn MLPClassifier

Observations and Comparison with Custom ReLU (Part d)

- **Test Performance:** The `scikit-learn` implementation achieved a **higher peak test F1-score (0.9098)** compared to our custom ReLU model's best (0.8914). As with our custom models, the 2-layer **{512, 256}** architecture performed the best.
- **Performance Drop-off:** Both implementations showed a performance degradation at 3 and 4 layers. However, the `scikit-learn` model's performance dropped more significantly at 4 layers (to 0.8728), whereas our custom ReLU implementation was more robust at greater depths (staying at 0.8911). This is likely due to the different stopping criteria; `sklearn`'s validation-based stopping halted training very early (14 iterations) as it detected overfitting, while our custom model continued training based on loss.
- **Training Convergence:** The `scikit-learn` models converged **dramatically faster**, stopping in 14-22 iterations. Our custom ReLU models, while much faster

than sigmoid, still required 23-70 epochs to converge. This highlights the high level of optimization within the `scikit-learn` library’s SGD solver.

- **Training Time:** The total training time was also significantly lower for the `scikit-learn` models. For instance, the 4-layer `sklearn` model finished in 312 seconds, while our custom 4-layer model took 421 seconds.

In conclusion, `scikit-learn`’s `MLPClassifier` was both faster and achieved a higher peak accuracy, demonstrating the effectiveness of its optimized and validation-based early stopping implementation. The 2-layer {512, 256} architecture was the optimal choice for both implementations.

Table 23: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Train Set** (`scikit-learn`).

Class	{512}			{512, 256}			{512, 256, 128}			{512, 256, 128, 64}		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	0.9933	0.9950	0.9942	0.9967	0.9967	0.9967	0.9693	1.0000	0.9844	0.9850	0.9833	0.9842
1	0.9934	0.9967	0.9950	0.9917	0.9900	0.9908	0.9949	0.9833	0.9891	0.9652	0.9717	0.9684
2	0.9835	0.9933	0.9884	0.9950	0.9917	0.9933	0.9901	0.9983	0.9942	0.9728	0.9550	0.9638
3	0.9900	0.9900	0.9900	0.9865	0.9750	0.9807	0.9704	0.9850	0.9777	0.9343	0.9000	0.9168
4	0.9917	0.9900	0.9908	0.9917	0.9950	0.9933	0.9899	0.9783	0.9841	0.9439	0.9817	0.9624
5	0.9884	0.9933	0.9909	0.9900	0.9917	0.9908	0.9899	0.9800	0.9849	0.9911	0.9267	0.9578
6	0.9849	0.9817	0.9833	0.9933	0.9817	0.9874	0.9817	0.9850	0.9834	0.9777	0.9517	0.9645
7	0.9967	0.9950	0.9958	0.9934	0.9983	0.9958	0.9851	0.9900	0.9875	0.9864	0.9667	0.9764
8	0.9967	0.9967	0.9967	0.9950	0.9967	0.9958	0.9950	0.9933	0.9942	0.9554	1.0000	0.9772
9	0.9868	0.9933	0.9900	0.9933	0.9883	0.9908	0.9966	0.9867	0.9916	0.9504	0.9900	0.9698
10	0.9901	0.9983	0.9942	0.9966	0.9917	0.9942	0.9884	0.9983	0.9934	1.0000	0.9750	0.9873
11	0.9950	0.9883	0.9916	0.9867	0.9917	0.9892	0.9900	0.9883	0.9892	0.9691	0.9917	0.9802
12	0.9884	0.9900	0.9892	0.9900	0.9900	0.9900	0.9834	0.9883	0.9859	0.9755	0.9283	0.9513
13	0.9933	0.9933	0.9933	0.9917	0.9933	0.9925	0.9965	0.9517	0.9736	0.9530	0.9800	0.9663
14	0.9900	0.9900	0.9900	0.9933	0.9917	0.9925	0.9950	1.0000	0.9975	0.9702	0.9767	0.9734
15	0.9950	0.9917	0.9933	0.9983	0.9950	0.9967	0.9900	0.9933	0.9917	0.9615	0.9983	0.9796
16	0.9916	0.9867	0.9891	0.9785	0.9883	0.9834	0.9882	0.9783	0.9832	0.9578	0.9083	0.9324
17	0.9917	0.9967	0.9942	0.9917	0.9917	0.9917	0.9461	0.9950	0.9699	0.9790	0.9333	0.9556
18	0.9916	0.9883	0.9900	0.9833	0.9833	0.9833	0.9767	0.9800	0.9784	0.9123	0.9883	0.9488
19	0.9933	0.9833	0.9883	0.9917	0.9933	0.9925	0.9915	0.9733	0.9823	0.9370	0.9667	0.9516
20	0.9852	0.9967	0.9909	0.9851	0.9950	0.9900	0.9899	0.9850	0.9875	0.9325	0.9217	0.9271
21	0.9917	0.9967	0.9942	1.0000	0.9950	0.9975	0.9950	1.0000	0.9975	0.9898	0.9750	0.9824
22	0.9849	0.9817	0.9833	0.9933	0.9850	0.9891	0.9931	0.9533	0.9728	0.8455	0.9850	0.9099
23	0.9867	0.9883	0.9875	0.9867	0.9917	0.9892	0.9916	0.9850	0.9883	0.9746	0.9583	0.9664
24	0.9866	0.9817	0.9841	0.9917	0.9917	0.9917	0.9770	0.9900	0.9834	0.9589	0.9733	0.9661
25	0.9882	0.9733	0.9807	0.9867	0.9917	0.9892	0.9834	0.9883	0.9859	0.9585	0.9250	0.9415
26	0.9950	0.9917	0.9933	0.9933	0.9900	0.9917	0.9966	0.9917	0.9942	0.9914	0.9617	0.9763
27	0.9917	0.9933	0.9925	0.9950	0.9917	0.9933	0.9950	0.9933	0.9942	0.9899	0.9800	0.9849
28	0.9721	0.9867	0.9793	0.9819	0.9950	0.9884	0.9506	0.9933	0.9715	0.9628	0.9500	0.9564
29	0.9932	0.9800	0.9866	0.9934	0.9983	0.9958	0.9933	0.9900	0.9917	0.9930	0.9467	0.9693
30	0.9819	0.9933	0.9876	0.9866	0.9850	0.9858	0.9851	0.9933	0.9892	0.8708	0.9883	0.9258
31	0.9802	0.9883	0.9842	0.9884	0.9900	0.9892	0.9752	0.9833	0.9793	0.9928	0.9133	0.9514
32	0.9933	0.9900	0.9917	0.9900	0.9933	0.9917	0.9948	0.9650	0.9797	0.9579	0.9850	0.9712
33	0.9933	0.9867	0.9900	0.9983	0.9883	0.9933	0.9916	0.9817	0.9866	0.9911	0.9317	0.9605
34	0.9866	0.9800	0.9833	0.9900	0.9900	0.9900	0.9835	0.9933	0.9884	0.9798	0.9700	0.9749
35	0.9899	0.9850	0.9875	0.9967	0.9983	0.9975	0.9983	0.9833	0.9908	0.9949	0.9733	0.9840

Table 24: Per-class Precision (P), Recall (R), and F1-Score (F1) on the **Test Set** (scikit-learn). Part 1 of 2.

Class	{512}			{512, 256}			{512, 256, 128}			{512, 256, 128, 64}		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	0.9632	0.9600	0.9616	0.9630	0.9533	0.9581	0.8731	0.9633	0.9160	0.9414	0.9100	0.9254
1	0.9097	0.8733	0.8912	0.9075	0.8833	0.8953	0.8974	0.9033	0.9003	0.8605	0.8433	0.8519
2	0.9164	0.9133	0.9149	0.9354	0.9167	0.9259	0.8903	0.9467	0.9176	0.9010	0.8800	0.8904
3	0.8367	0.8200	0.8283	0.8576	0.8633	0.8605	0.8185	0.8567	0.8371	0.7987	0.8200	0.8092
4	0.8878	0.8700	0.8788	0.9113	0.8567	0.8832	0.9288	0.8267	0.8748	0.8601	0.8200	0.8396
5	0.8889	0.9333	0.9106	0.9109	0.9200	0.9154	0.9128	0.9067	0.9097	0.9695	0.8467	0.9039
6	0.8953	0.8833	0.8893	0.8779	0.8867	0.8823	0.9091	0.9000	0.9045	0.9032	0.8400	0.8705
7	0.8940	0.9000	0.8970	0.8910	0.9267	0.9085	0.8966	0.9533	0.9241	0.9210	0.8933	0.9069
8	0.9561	0.9433	0.9497	0.9511	0.9733	0.9621	0.9599	0.9567	0.9583	0.8954	0.9700	0.9312
9	0.9439	0.9533	0.9486	0.9430	0.9367	0.9398	0.9279	0.9433	0.9355	0.9231	0.9200	0.9215
10	0.9604	0.9700	0.9652	0.9600	0.9600	0.9600	0.9353	0.9633	0.9491	0.9555	0.9300	0.9426
11	0.9344	0.9500	0.9421	0.9414	0.9633	0.9522	0.9279	0.9433	0.9355	0.9492	0.9333	0.9412
12	0.8874	0.8933	0.8904	0.8970	0.9000	0.8985	0.8940	0.9000	0.8970	0.8562	0.8333	0.8446
13	0.9121	0.9333	0.9226	0.9342	0.9467	0.9404	0.9429	0.8800	0.9103	0.8667	0.9100	0.8878
14	0.8762	0.9433	0.9085	0.9065	0.9367	0.9213	0.8981	0.9400	0.9186	0.8794	0.9233	0.9008
15	0.9286	0.9533	0.9408	0.9439	0.9533	0.9486	0.9381	0.9600	0.9489	0.8818	0.9700	0.9238
16	0.8481	0.8000	0.8233	0.8328	0.8467	0.8397	0.8571	0.8200	0.8382	0.8406	0.7733	0.8056
17	0.8893	0.8833	0.8863	0.8878	0.8700	0.8788	0.7994	0.9167	0.8540	0.9015	0.8233	0.8606
18	0.8214	0.8433	0.8322	0.8681	0.8333	0.8503	0.8767	0.8767	0.8767	0.7922	0.8767	0.8323
19	0.8958	0.8600	0.8776	0.8782	0.9133	0.8954	0.9259	0.8333	0.8772	0.8464	0.9000	0.8724
20	0.8599	0.9000	0.8795	0.8594	0.8967	0.8777	0.8962	0.8633	0.8795	0.8581	0.8267	0.8421
21	0.9568	0.9600	0.9584	0.9689	0.9333	0.9508	0.9336	0.9367	0.9351	0.9267	0.9267	0.9267
22	0.8409	0.8633	0.8520	0.8844	0.8667	0.8754	0.9574	0.7500	0.8411	0.6675	0.9100	0.7701
23	0.9069	0.8767	0.8915	0.9054	0.8933	0.8993	0.8881	0.8733	0.8807	0.8529	0.8700	0.8614
24	0.8938	0.8700	0.8818	0.8791	0.8967	0.8878	0.8140	0.8900	0.8503	0.8333	0.8833	0.8576
25	0.8467	0.8467	0.8467	0.8544	0.8800	0.8670	0.8131	0.8700	0.8406	0.8261	0.7600	0.7917
26	0.9597	0.9533	0.9565	0.9664	0.9600	0.9632	0.9689	0.9333	0.9508	0.9786	0.9167	0.9466
27	0.9213	0.9367	0.9289	0.9470	0.9533	0.9502	0.9619	0.9267	0.9440	0.9384	0.9133	0.9257
28	0.8552	0.8467	0.8509	0.8604	0.8833	0.8717	0.7952	0.8800	0.8354	0.8435	0.8267	0.8350
29	0.9070	0.9100	0.9085	0.9486	0.9233	0.9358	0.8885	0.9033	0.8959	0.9407	0.8467	0.8912
30	0.8850	0.9233	0.9038	0.8864	0.9100	0.8980	0.8667	0.9100	0.8878	0.7044	0.9533	0.8102
31	0.8945	0.8200	0.8557	0.9247	0.8600	0.8912	0.8416	0.8500	0.8458	0.9009	0.6967	0.7857
32	0.9170	0.8833	0.8998	0.9308	0.8967	0.9134	0.9338	0.8467	0.8881	0.8508	0.8933	0.8715
33	0.9106	0.9167	0.9136	0.9026	0.9267	0.9145	0.9223	0.8700	0.8954	0.9518	0.7900	0.8634
34	0.9139	0.9200	0.9169	0.9262	0.9200	0.9231	0.9167	0.9167	0.9167	0.8631	0.9033	0.8827
35	0.9007	0.9067	0.9037	0.9226	0.9133	0.9179	0.9708	0.8867	0.9268	0.9255	0.8700	0.8969

2.5 Transfer Learning for Digit Classification

In this final part, we explore transfer learning by adapting the model trained on the consonant dataset to classify the 10 Devanagari digits. We compare two approaches using the 4-layer ReLU network {512, 256, 128, 64}.

1. **Training from Scratch:** A 4-layer network was trained from scratch on the 10-class digit dataset for 20 epochs.
2. **Transfer Learning:** The 4-layer model pre-trained on the 36-class consonant dataset (from Part d) was loaded. Its final layer (64x36) was removed and replaced with a new, randomly initialized output layer (64x10). The entire network was then "fine-tuned" on the 10-class digit dataset for 20 epochs.

The training progress for both models is shown in Figure 17, and the final results are summarized in Table 25.

Observations

The results clearly demonstrate the benefit of transfer learning, even when the tasks are only moderately related (classifying different types of characters).

- **Head Start:** The most dramatic result is seen in the first epoch of training. The model trained from scratch starts with a test F1-score of 0.8999. In contrast, the fine-tuned model, leveraging its pre-trained weights, starts with a test F1-score of **0.9457**. This shows that the low-level features (lines, curves, edges) learned from the consonants are immediately useful for identifying digits.

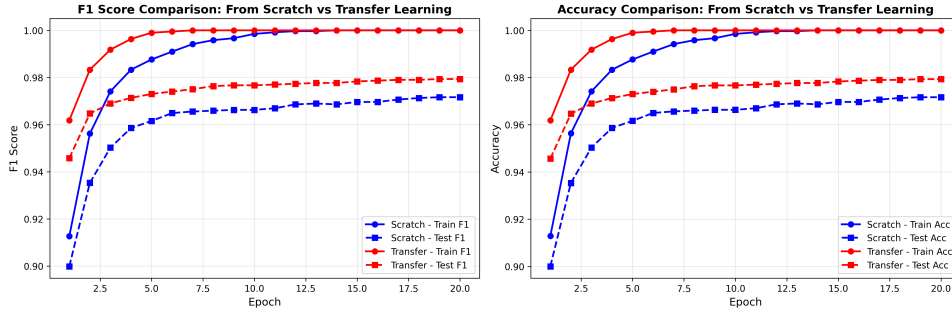


Figure 17: F1 Score and Accuracy Comparison: From Scratch vs. Transfer Learning. The transfer learning model (red) consistently outperforms the model trained from scratch (blue) on the test set.

Table 25: Final Comparison: Training from Scratch vs. Transfer Learning

Metric	From Scratch	Transfer Learning	Improvement
Final Test Accuracy	0.9717	0.9793	+0.0077
Final Test F1 Score	0.9716	0.9793	+0.0077
Final Train Accuracy	1.0000	1.0000	+0.0000
Final Train F1 Score	1.0000	1.0000	+0.0000
Training Time (sec)	102.59	124.89	+22.29

- **Better Final Performance:** The transfer learning model achieves a higher final test F1-score (0.9793) than the model trained from scratch (0.9716). This suggests that the pre-training helped the model converge to a better, more generalizable minimum in the loss landscape.
- **Faster Convergence:** As seen in Figure 17, the red line (Transfer) is consistently above the blue line (Scratch) for both training and testing. The transfer model achieves a test F1-score of 0.9713 by epoch 4, a level that the from-scratch model only reaches at epoch 19.
- **Training Time:** The fine-tuning model took slightly longer to train for the fixed 20 epochs (124.89s vs 102.59s). This is expected, as the model starts with more complex, non-zero weights, and the backpropagation calculations are more involved from the start, whereas the from-scratch model begins with small random weights.

In conclusion, transfer learning was highly successful. The features learned from the 36-class consonant set provided a significant performance boost, allowing the model to learn the 10-class digit set faster and achieve a better final result than training from scratch.